

Struktury danych	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>wtorek TN 11¹⁵ – 12⁵⁵</i>
Imię, nazwisko, numer albumu <i>Kamil Rachwałik 284174, Kamil Łupicki 284096</i>	Data <i>13.04.2026</i>
Link do projektu https://github.com/Permurez/Struktury_sem4	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 1

Spis treści

1	Wstęp	1
1.1	Wykonane struktury	1
1.2	Złożoność obliczeniowa	2
1.3	Zastosowania struktur	2
2	Warunki przeprowadzenia badań	3
3	Wyniki testów	4
3.1	Dodawanie na początku (addFront)	4
3.2	Dodawanie na końcu (addBack)	5
3.3	Dodawanie w losowym miejscu (addAtIndex rand)	6
3.4	Usuwanie z początku (removeFront)	7
3.5	Usuwanie z końca (removeBack)	8
3.6	Usuwanie z losowego miejsca (removeAtIndex rand)	9
3.7	Wyszukiwanie istniejącego elementu (find existing)	10
4	Wnioski	10

1 Wstęp

1.1 Wykonane struktury

W ramach miniprojektu zaimplementowano trzy dynamiczne struktury danych w języku C++:

- **Tablica dynamiczna (DynamicArray)** – przechowuje dane w ciągłym bloku pamięci. W momencie braku miejsca pojemność jest podwajana. Umożliwia szybki dostęp indeksowany, ale operacje wstawiania i usuwania poza końcem wymagają przesuwania elementów.
- **Lista jednokierunkowa (SinglyLinkedList)** – składa się z węzłów zawierających dane i wskaźnik na następny element. Przechowywane są wskaźniki na głowę (head) i ogon (tail), co pozwala na dodawanie na obu końcach w czasie $O(1)$. Usunięcie z końca wymaga przejścia całej listy ($O(n)$).
- **Lista dwukierunkowa (DoubleLinkedList)** – każdy węzeł zawiera wskaźniki na następny i poprzedni element. Dzięki temu operacje dodawania i usuwania zarówno na początku, jak i na końcu są wykonywane w czasie stałym.

1.2 Złożoność obliczeniowa

Tabela 1: Złożoność obliczeniowa operacji dla tablicy dynamicznej

Operacja	Optymistyczna	Średnia	Pesymistyczna
addFront	$O(n)$	$O(n)$	$O(n)$
addBack	$O(1)$	$O(1)$ (amort.)	$O(n)$
addAtIndex	$O(1)$	$O(n)$	$O(n)$
removeFront	$O(n)$	$O(n)$	$O(n)$
removeBack	$O(1)$	$O(1)$	$O(1)$
removeAtIndex	$O(1)$	$O(n)$	$O(n)$
find	$O(1)$	$O(n)$	$O(n)$

Tabela 2: Złożoność obliczeniowa operacji dla listy jednokierunkowej

Operacja	Optymistyczna	Średnia	Pesymistyczna
addFront	$O(1)$	$O(1)$	$O(1)$
addBack	$O(1)$	$O(1)$	$O(1)$
addAtIndex	$O(1)$	$O(n)$	$O(n)$
removeFront	$O(1)$	$O(1)$	$O(1)$
removeBack	$O(n)$	$O(n)$	$O(n)$
removeAtIndex	$O(1)$	$O(n)$	$O(n)$
find	$O(1)$	$O(n)$	$O(1) / O(n)$

Tabela 3: Złożoność obliczeniowa operacji dla listy dwukierunkowej

Operacja	Optymistyczna	Średnia	Pesymistyczna
addFront	$O(1)$	$O(1)$	$O(1)$
addBack	$O(1)$	$O(1)$	$O(1)$
addAtIndex	$O(1)$	$O(n)$	$O(n)$
removeFront	$O(1)$	$O(1)$	$O(1)$
removeBack	$O(1)$	$O(1)$	$O(1)$
removeAtIndex	$O(1)$	$O(n)$	$O(n)$
find	$O(1)$	$O(n)$	$O(n)$

1.3 Zastosowania struktur

- **Tablica dynamiczna** – podstawowa struktura dla kontenerów takich jak `std::vector`, stosowana w buforach danych, implementacji kolejek priorytetowych (kopiec) oraz wszędzie tam, gdzie dominuje szybki dostęp przez indeks i dodawanie na końcu.
- **Lista jednokierunkowa** – wykorzystywana w implementacjach stosów i kolejek FIFO (gdy operacje wykonywane są tylko na końcach), a także w systemach wbudowanych ze względu na mniejsze zużycie pamięci (jeden wskaźnik na węzeł). Może również testy na e-portalu gdzie nie można się cofać.
- **Lista dwukierunkowa** – stosowana w implementacjach kolejek dwukierunkowych (deque), mechanizmach „cofania” w edytorach tekstu, historii przeglądarki internetowej oraz wszędzie tam, gdzie wymagane jest poruszanie się po liście w obie strony.

2 Warunki przeprowadzenia badań

Eksperymenty wykonano na komputerze o następującej specyfikacji:

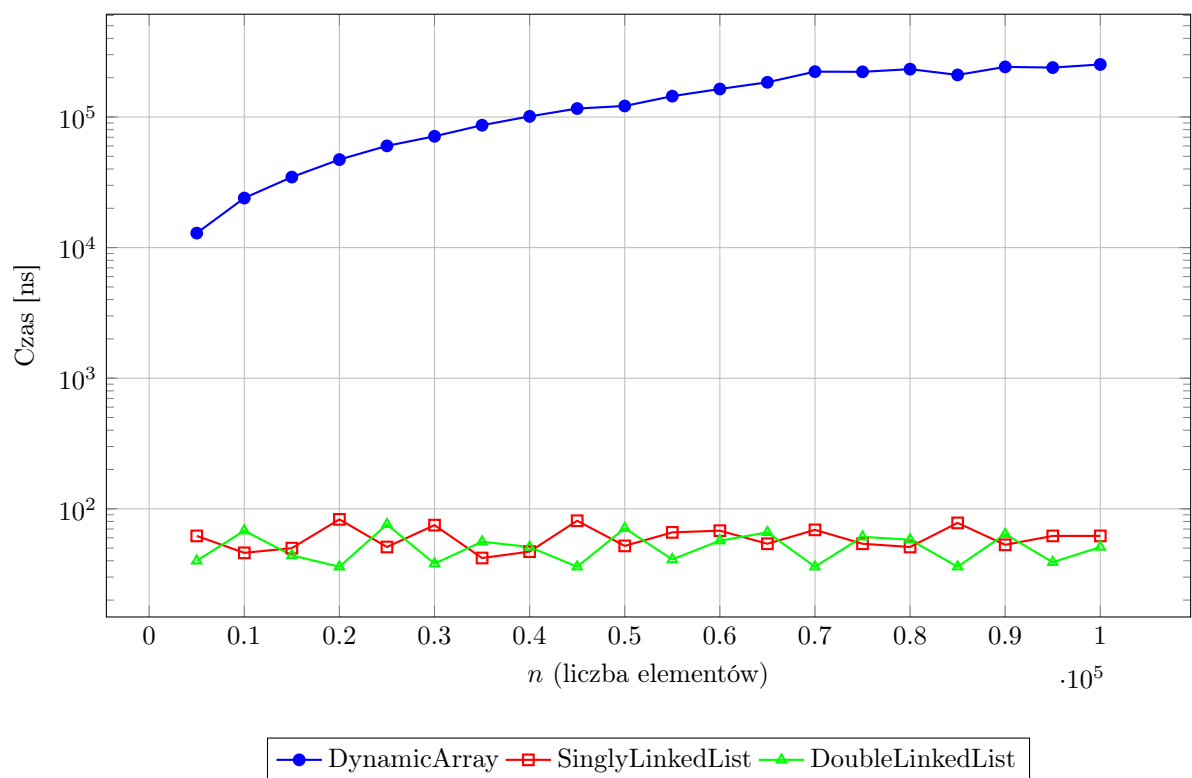
- Procesor: AMD Ryzen 5 3600x
- RAM: 32 GB DDR4
- System: Windows 10
- Kompilator: g++
- Sposób generowania danych: Dla każdego rozmiaru (n) tworzone od nowa trzy struktury danych: tablicę dynamiczną, listę jednokierunkową i listę dwukierunkową. Każda struktura była wypełniana losowymi liczbami całkowitymi wygenerowanymi na podstawie seedów. Następnie jeden losowo wybrany element zastępowano wartością 42, aby zapewnić obecność znanej wartości do testów wyszukiwania. Procedurę powtarzano niezależnie dla każdego punktu pomiarowego.
- Sposób pomiaru czasu: Pomiary wykonano dla ($n = 5000, 10000, 15000, \dots, 100000$) (20 punktów). Dla każdej operacji wykonywano serię 1000 wywołań, a mierzono łączny czas tej serii z użyciem `std::chrono::high_resolution_clock`. Wynik zapisywano w mikrosekundach i uśredniano z 30 powtórzeń, przy rotacji ziaren generatora pseudolosowego według reguły $(67 + (r \bmod 5))$ (seedy 67-71). Raportowana wartość to średni czas całej serii 1000 operacji podzielony przez 1000.

3 Wyniki testów

3.1 Dodawanie na początku (addFront)

Tabela 4: Czasy dodawania na początku [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	12913	62	40
10000	23957	46	68
20000	47201	83	36
30000	71192	75	38
40000	101112	47	51
50000	121478	52	71
60000	163922	68	57
70000	222560	69	36
80000	232528	51	58
90000	241979	53	64
100000	252575	62	51

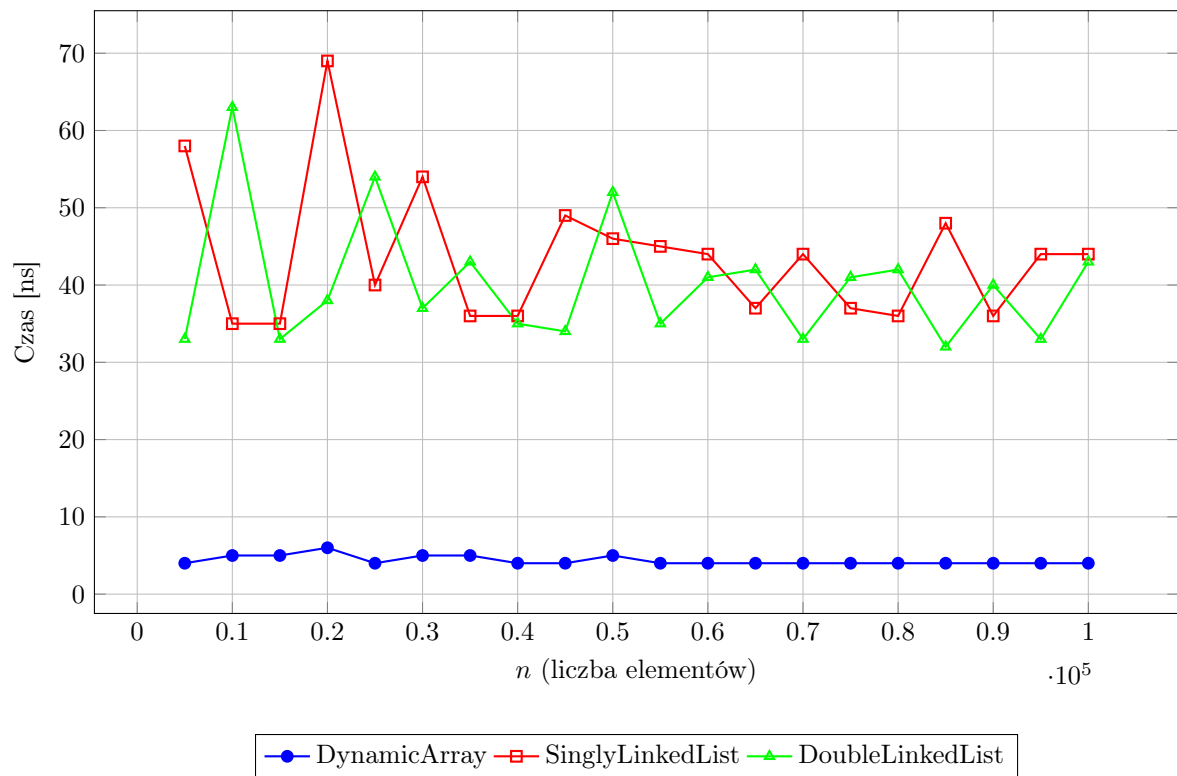


Rysunek 1: Wykres czasów dodawania na początku, listy nie muszą przesuwać elementów

3.2 Dodawanie na końcu (addBack)

Tabela 5: Czasy dodawania na końcu [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	4	58	33
10000	5	35	63
20000	6	69	38
30000	5	54	37
40000	4	36	35
50000	5	46	52
60000	4	44	41
70000	4	44	33
80000	4	36	42
90000	4	36	40
100000	4	44	43

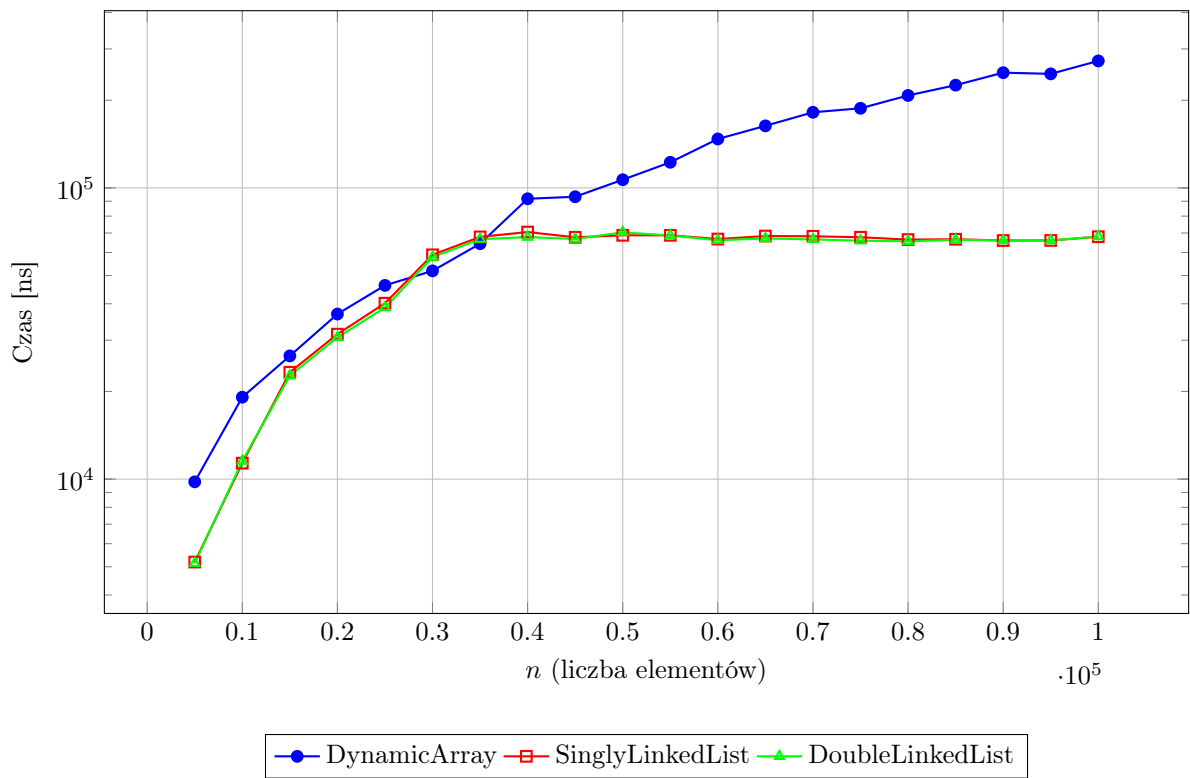


Rysunek 2: Wykres czasów dodawania na końcu.

3.3 Dodawanie w losowym miejscu (addAtIndex rand)

Tabela 6: Czasy dodawania w losowym indeksie [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	9786	5190	5142
10000	19116	11343	11532
20000	36880	31459	30650
30000	51898	58917	57822
40000	91760	70545	67774
50000	106668	68742	70377
60000	147243	66731	66128
70000	181840	68216	66558
80000	207710	66444	65523
90000	248692	65973	65994
100000	272927	67969	68081

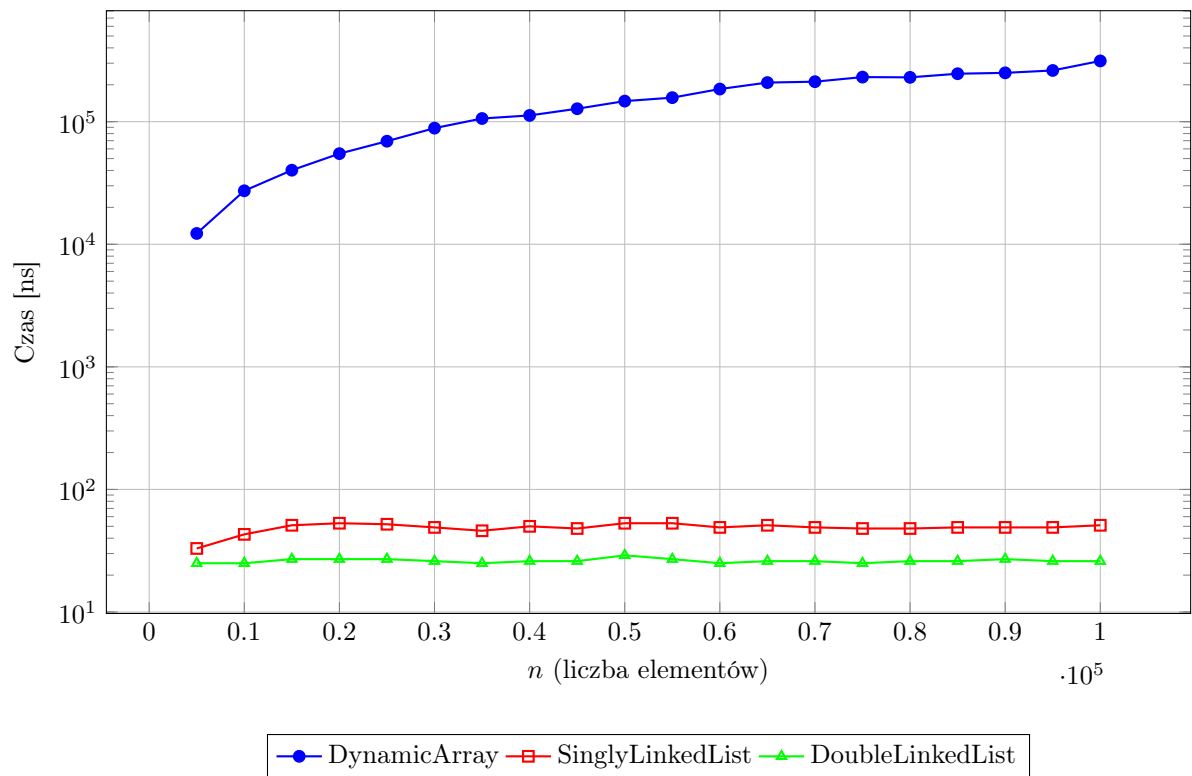


Rysunek 3: Wykres czasów dodawania w losowym miejscu (rand jest winny wypłaszczeniu po 32k)

3.4 Usuwanie z początku (removeFront)

Tabela 7: Czasy usuwania z początku [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	12250	33	25
10000	27309	43	25
20000	54867	53	27
30000	88597	49	26
40000	112444	50	26
50000	147172	53	29
60000	184636	49	25
70000	211953	49	26
80000	229958	48	26
90000	250215	49	27
100000	313019	51	26

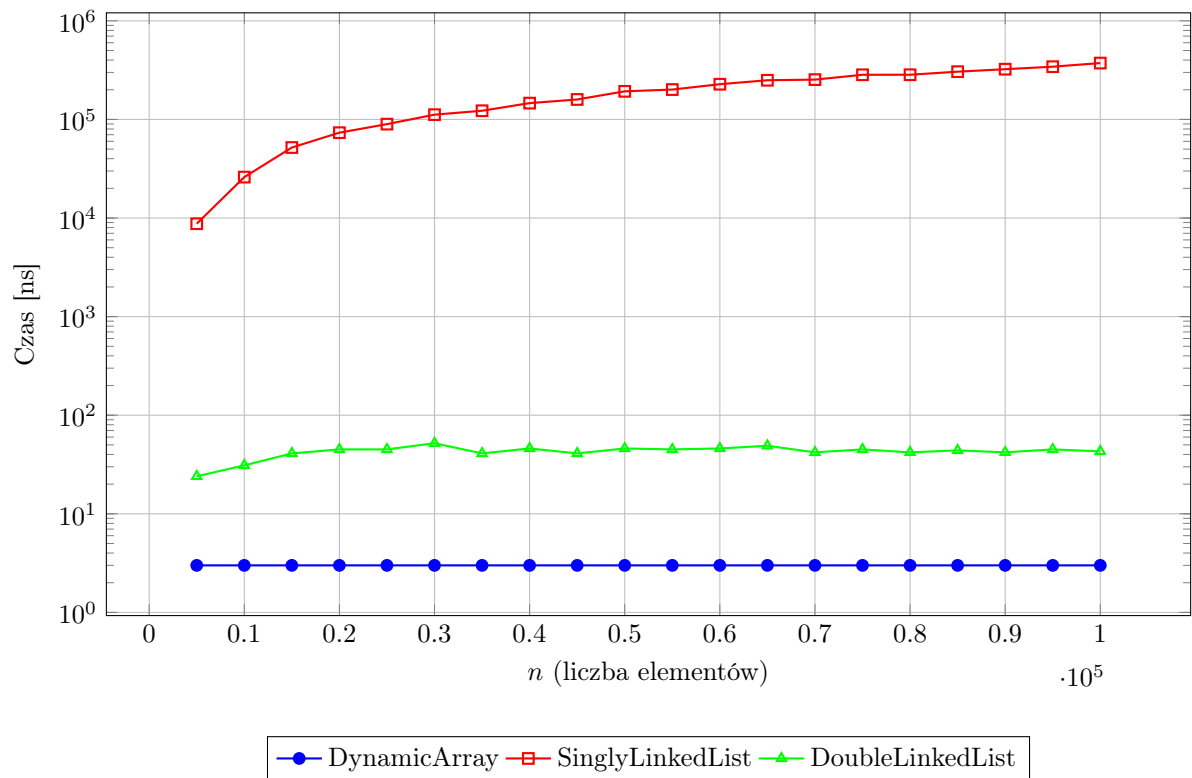


Rysunek 4: Wykres czasów usuwania z początku (oś Y logarytmiczna)

3.5 Usuwanie z końca (removeBack)

Tabela 8: Czasy usuwania z końca [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	3	8756	24
10000	3	26004	31
20000	3	73427	45
30000	3	111800	52
40000	3	146384	46
50000	3	192800	46
60000	3	227628	46
70000	3	253421	42
80000	3	284055	42
90000	3	323285	42
100000	3	373031	43

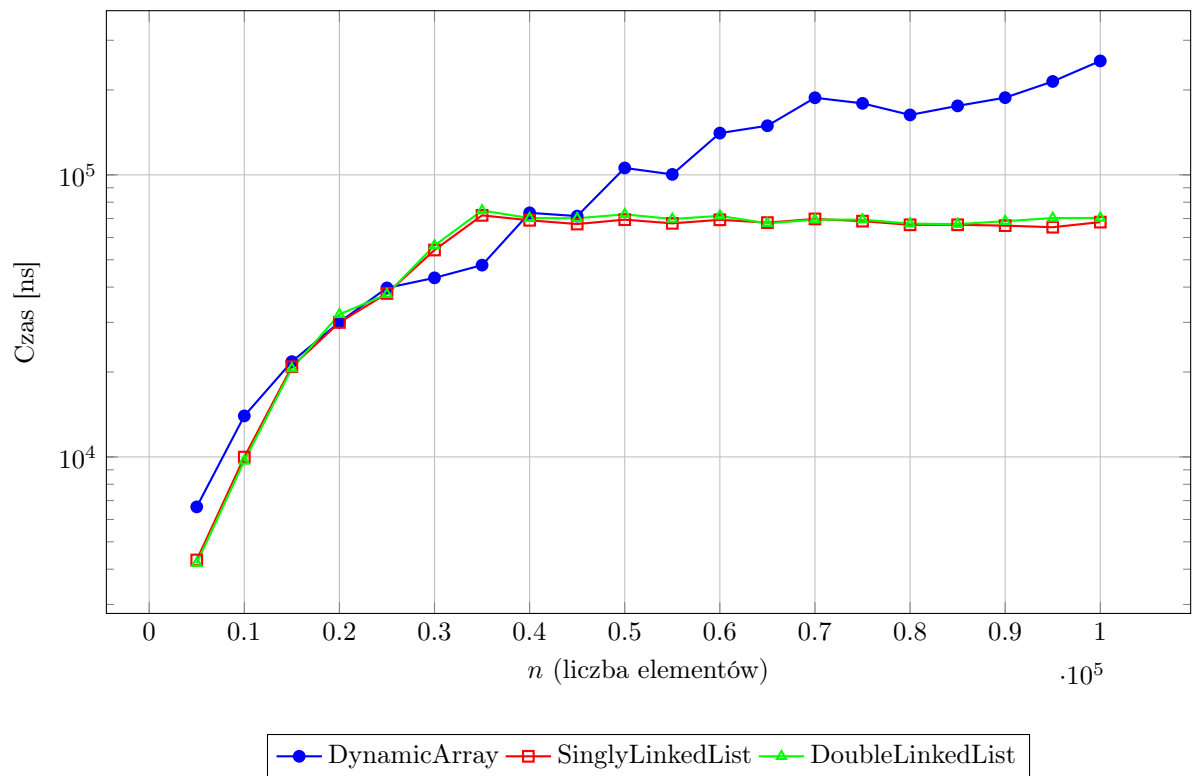


Rysunek 5: Wykres czasów usuwania z końca (oś Y logarytmiczna)

3.6 Usuwanie z losowego miejsca (removeAtIndex rand)

Tabela 9: Czasy usuwania z losowego indeksu [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	6652	4312	4200
10000	13977	9984	9678
20000	30043	29982	32022
30000	43140	54143	56149
40000	73343	69012	70266
50000	105797	69373	72447
60000	140629	69290	71554
70000	187640	69782	69377
80000	163098	66514	67149
90000	187816	66069	68505
100000	253443	67998	70345

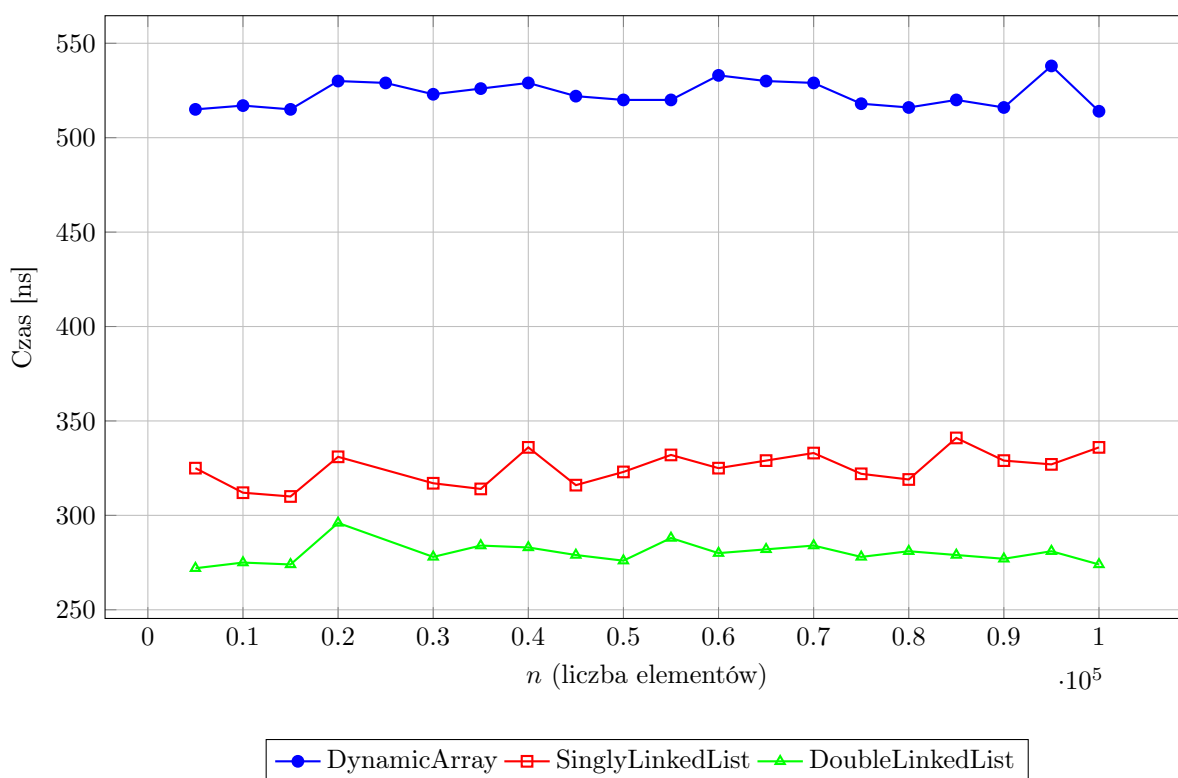


Rysunek 6: Wykres czasów usuwania z losowego miejsca (oś Y logarytmiczna)

3.7 Wyszukiwanie istniejącego elementu (find existing)

Tabela 10: Czasy wyszukiwania elementu 42 [ns]

n	DynamicArray	SinglyLinkedList	DoubleLinkedList
5000	515	325	272
10000	517	312	275
20000	530	331	296
30000	523	317	278
40000	529	336	283
50000	520	323	276
60000	533	325	280
70000	529	333	284
80000	516	319	281
90000	516	329	277
100000	514	336	274



Rysunek 7: Wykres czasów wyszukiwania (Stabilność czasów pokazuje przedwczesne pojawienie się wartości testowej 42)

4 Wnioski

- **Dodawanie i usuwanie na początku** – tablica dynamiczna: złożoność liniowa $O(n)$ (przesuwanie elementów). Listy (jedno- i dwukierunkowa): czas stały $O(1)$, niezależny od rozmiaru.
- **Dodawanie i usuwanie na końcu** – tablica dynamiczna (amortyzowane $O(1)$) i lista dwukierunkowa ($O(1)$) działają bardzo dobrze. Lista jednokierunkowa, mimo wskaźnika `tail`, usuwa z końca w czasie $O(n)$ – potwierdza to liniowy wzrost czasu na wykresie `removeBack`.
- **Operacje w losowym miejscu** – tablica dynamiczna wykazuje liniowy wzrost (konieczność przesuwania elementów). Dla list oczekiwano również $O(n)$, jednak dla dużych n czasy ustabilizowały

się (ok. 65 000 ns), co sugeruje wpływ pamięci podręcznej lub specyfiki pomiaru. Mimo to tablica pozostaje wyraźnie wolniejsza od list przy modyfikacjach w środku.

- **Wyszukiwanie** – wszystkie struktury: złożoność liniowa $O(n)$. Czasy zbliżone (ok. 500 ns dla tablicy, 270–330 ns dla list). Tablica nieznacznie wolniejsza mimo ciągłego bloku pamięci.
- **Porównanie ogólne** – tablica dynamiczna: najlepsza przy dostępie indeksowanym i operacjach na końcu, słaba przy modyfikacjach na początku/w środku. Lista jednokierunkowa: dobra dla operacji na końcach (z wyjątkiem usuwania z końca), oszczędna pamięciowo. Lista dwukierunkowa: uniwersalna, szybkie operacje na obu końcach, większy narzut pamięciowy.
- **Niezgodności z teorią i błędy metodologiczne** – Analiza wyników ujawniła dwa krytyczne punkty, w których dane odbiegają od teorii:
 - **Wyszukiwanie** (*find*): Uzyskane czasy (ok. 300–500 ns) są niemal stałe niezależnie od rozmiaru struktury. Jest to ewidentna niezgodność ze złożonością $O(n)$, wynikająca prawdopodobnie z błędów procedury testowej (prawdopodobnie jest to wina wypełniania tablicy losowymi elementami, liczba 42 musiała znajdować się w którymś z seedów wcześniej co zaburzyło wyniki i wykazało złożoność $O(1)$.
 - **Remove oraz Add at Index** czas tej operacji na listach nie rośnie po 32k prawdopodobnie poprzez błędne zastosowanie losowania elementu, czas `DynamicArray` dalej rośnie jak $O(n)$ poprzez to, że trzeba przesuwać wszystkie elementy po tym wylosowaniu.